



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	M. Rizki Algipari
NPM	2428240069
Kelas	SI5B
Topik Tugas 1	Topik 14 - Galeri Seni: Lukisan
Entitas utama	lukisan
Tautan repositori	https://github.com/Ridzz05/si2b_kuis1_mrizzkialgipari
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_2428240069_M. Rizki Algipari.pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Topik 14 - Galeri Seni: Lukisan
Entitas utama (tunggal)	lukisan
Alamat dasar	/paintings
Field data	id (number, otomatis), judul (string, wajib), pelukis (string, wajib), aliran (string, wajib), tahunDibuat (number, opsional), harga (number, wajib)
Field wajib saat POST	judul, pelukis, aliran, harga (tahunDibuat opsional)
Query string untuk filter	?aliran=<aliran>, contoh ?aliran=realisme
Data awal	3 data: id 1 Senja di Musi (realisme), id 2 Tari Cahaya Pagi (impresionisme), id 3 Bentuk Tanpa Nama (abstrak)
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/paintings	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/paintings/:id	Tidak	200	Ambil satu data
3	POST	/paintings	Ya	201	Tambah data
4	PUT	/paintings/:id	Ya	200	Ubah data
5	DELETE	/paintings/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah node -v, npm -v, npm init -y, dan npm install express cors dotenv beserta keluarannya sampai selesai.

```

muhri@muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ node -v
v24.21.0
muhri@muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ npm -v
11.19.0
muhri@muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ npm init -y
Wrote to C:\Users\muhri\Documents\tugas1-restful-2428240069\package.json:
muhri@muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ npm install express cors dotenv

up to date in 414ms
muhri@muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ cat package.json
{
  "name": "tugas1-restful-2428240069",
  "version": "1.0.0",
  "description": "RESTful API Galeri Seni - Lukisan (Topik 14)",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "keywords": [],
  "author": "M. Rizki Algipari",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^16.6.1",
    "express": "^5.2.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.9"
  }
}

```

Gambar 1. Versi Node.js dan instalasi paket

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi express, cors, dan dotenv, serta mengapa versi Node.js perlu memenuhi syarat Express 5.

Di app.js, require("dotenv").config() dipanggil paling atas supaya variabel pada .env, misalnya API_KEY, terbaca lewat process.env. express dipakai untuk membuat server dan mendaftarkan rute, contohnya app.get("/", ...) dan app.use("/paintings", paintingsRoutes). cors dipasang lewat app.use(cors()) agar permintaan dari origin lain seperti Postman tidak diblokir, sedangkan express.json() mengubah body JSON menjadi objek. Express 5 mensyaratkan Node.js minimal versi 18, sedangkan di komputer saya terpasang Node v24.21.0, jadi syarat tersebut sudah terpenuhi.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ ls -R models controllers routes middlewares
controllers:
paintingsController.js

middlewares:
cekApiKey.js
errorHandler.js
logger.js

models:
paintingsModel.js

routes:
paintingsRoutes.js

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ ls --ignore=node_modules
.env
.env.example
.gitignore
.kilo
.omp
INSTRUKSI
Laporan_Tugas1_2428240069_M Rizki Alqipari.docx
README.md
app.js
controllers
middlewares
models
package-lock.json
package.json
postman_collection.json
routes
screenshots
vercel.json

```

Gambar 2. Struktur folder project

Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

Folder routes menyimpan router yang mengarahkan alamat ke controller, controllers memuat logika permintaan dan validasi, models menyimpan data beserta fungsi aksesnya, dan middlewares memuat fungsi yang berjalan di antara permintaan dan controller.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

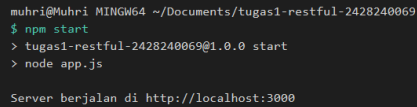
Berkas	Penjelasan Anda
models/paintingsModel.js	Menyimpan array paintings berisi tiga lukisan awal dan variabel nextId untuk penomoran id otomatis. Berisi fungsi getAllPaintings(aliran), getPaintingById(id), createPainting(data), updatePainting(id, data), dan deletePainting(id).
controllers/paintingsController.js	Berisi validasiLukisan(body) yang memeriksa field wajib, lalu fungsi getAll, getById, create, update, dan remove yang memanggil model dan mengirim respons.

Berkas	Penjelasan Anda
routes/paintingsRoutes.js	Mendaftarkan lima rute lewat <code>express.Router()</code> : <code>GET /</code> , <code>GET /:id</code> , <code>POST /</code> , <code>PUT /:id</code> , dan <code>DELETE /:id</code> . Rute <code>POST</code> , <code>PUT</code> , dan <code>DELETE</code> dipasang <code>middleware cekApiKey</code> .
middlewares/logger.js	Fungsi <code>logger</code> mencatat tiap permintaan lewat <code>res.on("finish", ...)</code> dan menulis <code>req.method</code> , <code>req.originalUrl</code> , serta <code>res.statusCode</code> , lalu memanggil <code>next()</code> .
middlewares/cekApiKey.js	Fungsi <code>cekApiKey</code> membandingkan <code>req.get("x-api-key")</code> dengan <code>process.env.API_KEY</code> . Bila tidak cocok, dikirim respons 401 dan <code>next()</code> tidak dipanggil.
middlewares/errorHandler.js	Berisi <code>notFound</code> untuk endpoint tak dikenal (404) dan <code>errorHandler</code> yang mengubah JSON rusak menjadi 400 "Format JSON tidak valid", selain itu membalas 500.
app.js	Entry point yang memanggil <code>require("dotenv").config()</code> , lalu <code>app.use(cors())</code> , <code>app.use(express.json())</code> , dan <code>app.use(logger)</code> , memasang <code>app.use("/paintings", paintingsRoutes)</code> , ditutup <code>app.use(notFound)</code> dan <code>app.use(errorHandler)</code> .

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ npm start
> tugas1-restful-2428240069@1.0.0 start
> node app.js

Server berjalan di http://localhost:3000
```

Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Nomor port diambil dari `process.env.PORT` dengan cadangan 3000, sesuai baris `const PORT = process.env.PORT || 3000` pada `app.js`, sehingga server berjalan di `http://localhost:3000`. Bila port itu sudah dipakai aplikasi lain, Node melempar error `EADDRINUSE` dan server gagal menyala, jadi aplikasi tersebut harus ditutup atau portnya diganti.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /paintings)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /paintings
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON berisi 3 objek lukisan.
Status hasil uji	200 OK; larik berisi 3 objek
Lulus?	[x] Ya [] Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i http://localhost:3000/paintings
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 357
ETag: W/"165-hGuhLDcilwFa1VOnl55j582jlc"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"judul":"Senja di Musi","pelukis":"Rahmat Hidayat","aliran":"realisme","tahunDibuat":2023,"harga":7500000}, {"id":2,"judul":"Tari Cahaya Pagi","pelukis":"Siti Nurhaliza","aliran":"impresionisme","tahunDibuat":2021,"harga":5200000}, {"id":3,"judul":"Bentuk Tanpa Nama","pelukis":"Bagas Prasetyo","aliran":"abstrak","tahunDibuat":2024,"harga":9800000}]

```

Gambar 4. Hasil GET semua data

Gambar 4. Hasil GET semua data

Penjelasan:

Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Endpoint ini membalas 200 OK karena seluruh data berhasil diambil. Urutannya: rute router.get("/", controller.getAll) di paintingsRoutes.js memanggil fungsi getAll di paintingsController.js, lalu getAll memanggil model.getAllPaintings(req.query.aliran) yang mengembalikan isi array paintings.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /paintings/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id 1.
Status hasil uji	200 OK; objek dengan id 1
Lulus?	[x] Ya [] Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i http://localhost:3000/paintings/1
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 114
ETag: W/"72-RcbDQFNulfg0p1MVtIQ2FGukvMA"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"judul":"Senja di Muli","pelukis":"Rahmat Hidayat","aliran":"realisme","tahunDibuat":2023,"harga":7500000}

```

Gambar 5. Hasil GET berdasarkan ID

Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

Jelaskan bagaimana ID dari alamat diambil (*req.params*) dan mengapa perlu diubah menjadi angka.

Nilai id diambil dari *req.params.id* yang masih berupa string, lalu diubah memakai `Number(req.params.id)` di fungsi `getById` pada `paintingsController.js`. Konversi ini perlu karena id pada model `paintings` disimpan sebagai angka, sehingga pencarian `p.id === id` di `getPaintingById` dapat cocok.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /paintings
Header	x-api-key: rahasia2428240069 Content-Type: application/json
Body	{ "judul": "...", "pelukis": "..." }
Hasil yang diharapkan	201 Created; objek baru dengan id 4 dan pesan "Data berhasil ditambahkan".
Status hasil uji	201 Created; pesan "Data berhasil ditambahkan", objek dengan id 4
Lulus?	[x] Ya [] Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i -X POST -H "x-api-key: rahasia2428240069" -H "Content-Type: application/json" -d '{"judul":"Senja di Musi","pelukis":"M. Rizki Algipari","aliran":"realisme","tahunDibuat":2023,"harga":7500000}' http://localhost:3000/paintings
HTTP/1.1 201 Created
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 183
ETag: W/"b7-MEib+WebP/nAXDukx+vNJF3HnFc"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","message":"Data berhasil ditambahkan","data":{"id":4,"judul":"Senja di Musi","pelukis":"M. Rizki Algipari","aliran":"realisme","tahunDibuat":2023,"harga":7500000}}

```

Gambar 6. Hasil POST data baru

Gambar 6. Hasil POST data baru

Penjelasan:

Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

Header `x-api-key` diperiksa middleware `cekApiKey` sebelum permintaan sampai ke controller, jadi kunci yang salah ditolak lebih dulu. `express.json()` yang dipasang lewat `app.use(express.json())` mengubah body JSON menjadi objek agar validasi `Lukisan` dapat membacanya. Status yang dipakai 201 Created, bukan 200, karena server berhasil membuat data baru dan memberi id dari `nextId`.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /paintings/4
Header	x-api-key: rahasia2428240069 Content-Type: application/json
Body	{ "judul": "Senja di Musi (Revisi)" }
Hasil yang diharapkan	200 OK; objek dengan judul "Senja di Musi (Revisi)".
Status hasil uji	200 OK; pesan "Data lukisan dengan id 4 berhasil diperbarui", judul "Senja di Musi (Revisi)"
Lulus?	☒ Ya ☐ Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i -X PUT -H "x-api-key: rahasia2428240069" -H "Content-Type: application/json" -d '{"judul":"Senja di Musi (Revisi)","pelukis":"Rahmat Hidayat","aliran":"impresionisme","tahunDibuat":2024,"harga":8000000}' http://localhost:3000/paintings/4
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 213
ETag: W/"d5-IRFYuaHjMdIKDY75aSAaI2PC7Y"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","message":"Data lukisan dengan id 4 berhasil diperbarui","data":{"id":4,"judul":"Senja di Musi (Revisi)","pelukis":"Rahmat Hidayat","aliran":"impresionisme","tahunDibuat":2024,"harga":8000000}}

```

Gambar 7. Hasil PUT mengubah data

Gambar 7. Hasil PUT mengubah data

Penjelasan:

Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

Field judul berubah menjadi "Senja di Musi (Revisi)", sedangkan pelukis, aliran, tahunDibuat, dan harga tetap. Perubahan dikerjakan fungsi updatePainting(id, data) pada paintingsModel.js yang mengganti isi array paintings pada index yang cocok, lalu controller update mengirim respons 200.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /paintings/4
Header	x-api-key: rahasia2428240069
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content; tanpa body
Lulus?	[x] Ya [] Tidak

```
muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i -X DELETE -H "x-api-key: rahasia2428240069" http://localhost:3000/paintings/4
HTTP/1.1 204 No Content
X-Powered-By: Express
Access-Control-Allow-Origin: *
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Gambar 8. Hasil DELETE data

Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Status 204 No Content berarti permintaan berhasil tetapi tidak ada isi yang perlu dikirim kembali. Karena itu fungsi `remove` di `paintingsController.js` memanggil `res.status(204).end()` tanpa body, dan tidak ada JSON yang tampil.

Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /paintings (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan "API key tidak valid atau tidak ditemukan".
Status hasil uji	401 Unauthorized; pesan "API key tidak valid atau tidak ditemukan"
Lulus?	[x] Ya [] Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i -X POST -H "Content-Type: application/json" -d '{"judul":"x","pelukis":"y","aliran":"z","harga":1}' http://localhost:3000/paintings
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 83
ETag: W/"53-tA1JUcbfjL1o5z120Z8g07NB7sc"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"API key tidak valid atau tidak ditemukan","data":null}

```

Gambar 9. POST tanpa API key ditolak

Gambar 9. POST tanpa API key ditolak

Penjelasan:

Jelaskan middleware mana yang menolak permintaan dan mengapa Controller tidak sempat dijalankan.

Middleware cekApiKey yang menolak permintaan ini karena req.get("x-api-key") bernilai undefined dan tidak sama dengan process.env.API_KEY. Fungsi create di controller tidak sempat berjalan sebab cekApiKey langsung membalas 401 tanpa memanggil next().

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /paintings
Header	x-api-key: rahasia2428240069 Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan "Field judul wajib diisi".
Status hasil uji	400 Bad Request; pesan "Field judul wajib diisi"
Lulus?	☒ Ya ☐ Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i -X POST -H "x-api-key: rahasia2428240069" -H "Content-Type: application/json" -d '{"pelukis":"y","aliran":"z","harga":1}' http://localhost:3000/pain
tings
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 66
ETag: W/"42-XrUvAkJOez2jTxbez6rQcphW3hI"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"Field judul wajib diisi","data":null}

```

Gambar 10. POST dengan body tidak lengkap

Gambar 10. POST dengan body tidak lengkap

Penjelasan:

Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

Validasi ditulis di controller, tepatnya pada fungsi validasiLukisan(body) di paintingsController.js. Karena body tidak memuat judul, fungsi itu mengembalikan pesan "Field judul wajib diisi", lalu create membalas 400 Bad Request.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /paintings
Header	x-api-key: rahasia2428240069 Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid".
Status hasil uji	400 Bad Request; pesan "Format JSON tidak valid"
Lulus?	☒ Ya ☐ Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i -X POST -H "x-api-key: rahasia2428240069" -H "Content-Type: application/json" -d '{"judul":"x","pelukis":"y",,}' http://localhost:3000/paintings
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 66
ETag: W/"42-c55eYsszNeCZmwM1xK59rYbMYdM"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"Format JSON tidak valid","data":null}

```

Gambar 11. POST dengan JSON rusak

Gambar 11. POST dengan JSON rusak

Penjelasan:

Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

JSON rusak dideteksi `express.json()` yang melempar `SyntaxError`, lalu ditangkap `errorHandler` di `middlewares/errorHandler.js`. `errorHandler` memeriksa `err instanceof SyntaxError` dan `err.status === 400`, kemudian mengubahnya menjadi respons rapi berisi 400 dengan pesan "Format JSON tidak valid".

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /paintings/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan "Data dengan id 99 tidak ditemukan".
Status hasil uji	404 Not Found; pesan "Data dengan id 99 tidak ditemukan"
Lulus?	☒ Ya ☐ Tidak

```

muhri@Muhri MINGW64 ~/Documents/tugas1-restful-2428240069
$ curl -s -i http://localhost:3000/paintings/99
HTTP/1.1 404 Not Found
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 76
ETag: W/"4c-rMUh+y54S396aKswIwB/bvt+xYs"
Date: Thu, 08 Oct 2026 05:55:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"Data dengan id 99 tidak ditemukan","data":null}

```

Gambar 12. GET ID tidak ada

Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Permintaan GET /paintings/99 masuk ke fungsi getByld di paintingsController.js yang memanggil model.getPaintingByld(99). Karena tidak ada lukisan berid 99, hasilnya null, lalu controller mengirim res.status(404).json berisi pesan "Data dengan id 99 tidak ditemukan".

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Ya
T02	Menampilkan satu data berdasarkan ID	200	200	Ya
T03	Menambah data baru	201	201	Ya
T04	Mengubah data	200	200	Ya
T05	Menghapus data	204	204	Ya
N01	Menambah data tanpa API key	401	401	Ya
N02	Menambah data dengan body tidak lengkap	400	400	Ya
N03	Mengirim JSON yang rusak	400	400	Ya
N04	Mengambil ID yang tidak ada	404	404	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/Ridzz05/si2b_kuis1_mrzkialgipari
Pesan commit terakhir	docs: tambah dokumen PDF dokumentasi kuis refactor
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Tulis jawaban Anda di sini.

Route bertugas mengarahkan permintaan ke fungsi yang tepat, contohnya `router.get("/", controller.getAll)` pada `paintingsRoutes.js`. Controller mengatur logika dan bentuk respons, contohnya `getAll` yang memanggil `res.status(200).json(model.getAllPaintings(req.query.aliran))`. Model menyimpan serta mengakses data, contohnya `getAllPaintings(aliran)` yang mengembalikan array `paintings` atau hasil filter berdasarkan aliran.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Tulis jawaban Anda di sini.

Error handler terpusat menangani kegagalan di satu tempat sehingga penanganan error tidak perlu ditulis ulang di tiap route. Buktinya pada pengujian N03, JSON rusak tidak membuat server berhenti karena `errorHandler` mengubahnya menjadi 400 "Format JSON tidak valid", dan pada N04 pesan 404 "Data dengan id 99 tidak ditemukan" tetap keluar rapi dari controller `getByld`, sedangkan endpoint yang benar-benar tidak dikenal ditangani `notFound`.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	✓
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	✓
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	✓
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	✓
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100